IA Aplicada com LLMs

Aula 13 — De script a imagem

Roteiro

1. **O que muda quando o agente sai da máquina** — imagem, container e o que fica de fora
2. **A imagem** — o `Dockerfile` do agente, camadas, dependências e contexto de build
3. **Construir e executar** — a *tag* da imagem, o modelo fora do container, o agente como API e o segredo, duas vezes
4. **O container é efêmero** — onde mora o estado, e o banco ao lado da API no `docker compose`
5. **Fecho** — a entrega de execuções longas, como conceito, e a superfície para a aula de segurança

Objetivos

Ao final deste deck, espera-se que o aluno seja capaz de:

- **distinguir** imagem e container, e **identificar** o que fica fora da imagem: segredo, modelo e estado;
- **empacotar** o agente numa imagem reprodutível, em quatro blocos: base fixada, dependências antes do código, `COPY` por nome e o comando — sem segredo em camada;
- **executar** o agente em container — na linha de comando e como API —, com a imagem identificada pela *tag* e o segredo entregue em tempo de execução;
- **decidir** onde mora o estado de um sistema cujo container é efêmero, e **externalizá-lo** num banco subido pelo `docker compose`.

Parte 1

O que muda quando o agente sai da máquina

O artefato que viaja é a imagem, não o código

O agente que funciona numa máquina depende do que aquela máquina tem: a versão do Python, os pacotes instalados, as variáveis de ambiente, o sistema operacional. Copiar o `agente.py` não copia nada disso.

	Imagem	**Container**
o que é	sistema de arquivos **imutável** em camadas, com os metadados de execução (diretório de trabalho, porta, comando)	uma **execução** da imagem: um processo isolado sobre uma cópia gravável do sistema de arquivos dela
quando nasce	uma vez, no `docker build`	a cada `docker run`
identidade	a *tag* — `auditor-de-despesas:1.0.0`	um identificador efêmero
quando termina	não termina; é promovida entre ambientes sem mudar	com o processo; com `--rm`, é apagado na hora

Imagem, container e `Dockerfile` foram apresentados na nota de Docker, na aula sobre o curso; aqui são aplicados ao agente. O agente é deliberadamente pequeno — um auditor de despesas com duas ferramentas, montado com `create_agent` —, porque o assunto é a imagem que o carrega. Ele aparece em três formas, cada uma numa pasta que é também o contexto de build: `01-agente-cli/` (programa), `02-agente-web/` (API) e `03-agente-web-banco/` (API com a política numa tabela do Postgres).

Três coisas ficam fora da imagem

Fora da imagem	Por quê	Como chega ao container	Onde é tratado
segredo	quem tem a imagem teria a chave	em tempo de execução, `--env-file .env`	Parte 3
modelo	é um provedor de API ou um servidor (Ollama) fora do container	pela rede: chave de API, ou endereço da máquina hospedeira	Parte 3
estado	o sistema de arquivos do container nasce da imagem a cada execução	por endereço declarado: volume, banco, serviço — ou não chega	Parte 4

A imagem carrega **código e dependências**, e nada que mude entre execuções ou entre quem as executa. É essa separação que permite construir uma vez e executar igual em qualquer máquina com Docker.

Parte 2

A imagem

O `Dockerfile` do agente, em quatro blocos

`01-agente-cli/Dockerfile`, sem os comentários longos do arquivo:

```
# 1  base com versão FIXADA até o patch
FROM python:3.11.13-slim-bookworm
WORKDIR /app

# 2  dependências ANTES do código
COPY requirements.txt .
RUN pip install -r requirements.txt

# 3  só os módulos do agente, por nome
COPY agente.py cliente.py ./

# 4  o programa fixo; os argumentos do docker run viram o objetivo
ENTRYPOINT ["python", "agente.py"]
```

No `Dockerfile`, comentário só vale em linha própria: um `#` no fim de uma instrução vira argumento dela. Os três `Dockerfile` da aula têm os mesmos blocos 1 a 3; o bloco 4 é o que distingue o programa da API.

A razão de cada bloco

#	Bloco	Razão
1	`FROM python:3.11.13-slim-bookworm` + `WORKDIR /app`	`python:3.11-slim` é etiqueta móvel: a mesma linha produz imagens diferentes em dias diferentes — o `-latest` da aula de escolha de modelos, no ambiente. `WORKDIR` fixa o diretório em que os blocos seguintes copiam e o processo executa
2	`COPY requirements.txt .` + `RUN pip install`	a instalação é a camada mais cara e o código é o que muda a cada *commit*: com o `requirements.txt` copiado sozinho, a camada só é refeita quando as dependências mudam
3	`COPY` com lista explícita	um `COPY . .` dependeria só do `.dockerignore` para manter o `.env` fora; a lista por nome é a segunda barreira
4	o comando	`ENTRYPOINT` fixa o programa, e `docker run <imagem> "<objetivo>"` passa o objetivo como argumento; nenhum segredo aqui — a chave chega em tempo de execução

O processo roda como `root`, o padrão do Docker. É uma das propriedades da imagem que a aula de segurança examina.

Camadas: a ordem decide o que é reconstruído

Cada instrução que altera o sistema de arquivos produz uma **camada**. A camada i_k sai do **cache de build** se, e somente se, a instrução e os arquivos que ela lê não mudaram **e** $i_1, \ldots, i_{k-1}$ também saíram do cache. Uma mudança que invalida i_k custa

$$T_{\text{build}}(k) = \sum_{j=k}^{n} t_j$$

Build depois de alterar uma linha do `agente.py`:

Passo	Ordem errada	Ordem do `Dockerfile` da aula
1	`COPY . .` — refeita	`COPY requirements.txt .` — cache
2	`RUN pip install` — **refeita**, dezenas de segundos a minutos	`RUN pip install` — cache
3	—	`COPY agente.py cliente.py ./` — refeita, menos de 1 s

O que muda pouco vem antes do que muda muito. Demonstração: alterar uma linha do agente e reconstruir — o `pip install` aparece como `CACHED`.

Só o que o agente importa, só o que o contexto contém

`requirements.txt` **próprio**, em cada pasta, com versão exata — e não o da disciplina inteira: imagem menor, build mais rápido, menos código de terceiro dentro do container. Na `01-agente-cli/`, nove pacotes: `langchain==1.3.15`, `langchain-core==1.5.4`, `langgraph==1.2.11`, as integrações dos três provedores (`langchain-mistralai`, `langchain-ollama` e `ollama`, `langchain-groq` e `groq`) e `python-dotenv==1.2.2`. A `02-agente-web/` acrescenta `fastapi` e `uvicorn`; a `03-agente-web-banco/`, o driver `psycopg[binary]`, que traz a `libpq` compilada e dispensa compilador na imagem *slim*.

Contexto de build é o diretório cujo conteúdo o construtor pode ler — aqui, a pasta de cada agente (o `.` do `docker build`). O que não entra no contexto **não pode ser copiado** por nenhuma instrução. O `.dockerignore` exclui do contexto:

```
.env    .env.*    !.env.example    *.key    *.pem
.git    .venv     __pycache__      *.pyc
```

A mesma lista do `.gitignore`, pela mesma razão. **Duas barreiras independentes** mantêm o `.env` fora da imagem: o `.dockerignore` e o `COPY` com lista explícita.

Parte 3

Construir e executar

Construir com uma *tag*, executar com o segredo de fora

```
docker build -t auditor-de-despesas:1.0.0 .
docker run --rm --env-file .env auditor-de-despesas:1.0.0 "A despesa D-4613 está dentro da política?"
```

```
auditor-de-despesas · mistral/mistral-small-latest
objetivo: A despesa D-4613 está dentro da política?
...
```

- `-t auditor-de-despesas:1.0.0` dá à imagem nome e *tag*; o `.` é o contexto de build. A *tag* identifica a imagem — é por ela que o `docker run`, o `docker history` e o registro a referenciam —, e o programa não a conhece.
- A primeira linha da saída vem do `agente.py`: o nome do sistema e o provedor e modelo resolvidos pelo `cliente.py`.
- `--env-file .env` entrega o segredo **em tempo de execução**: ele não passa pelo build e não fica em camada nenhuma.
- `--rm` apaga o container quando o processo termina; a imagem continua.
- O build não chama o modelo; cada `docker run` é uma execução do agente.

Dentro do container, `localhost` é o próprio container

Provedor (`LLM_PROVEDOR`)	O que o container precisa
`mistral`, `groq` — API	só a chave, que chega pelo `--env-file`
`ollama` — servidor na máquina hospedeira	o **endereço da máquina**, não o do container

O `.env.example` traz `OLLAMA_BASE_URL=http://localhost:11434`, correto fora do container. Dentro dele, `localhost` é a interface de *loopback* do próprio container, onde nada escuta na porta 11434. O endereço é sobrescrito no `docker run`:

```
docker run --rm --env-file .env -e OLLAMA_BASE_URL=http://host.docker.internal:11434 auditor-de-despesas:1.0.0
```

`host.docker.internal` é o nome que o Docker resolve para a máquina hospedeira. O `-e` prevalece sobre a mesma variável do `--env-file`. Com `validate_model_on_init=True`, o `cliente.py` falha já na partida, com mensagem clara, se o modelo não for alcançado.

O mesmo agente como API: muda o bloco 4

`02-agente-web/`: o `api.py` (FastAPI) chama o mesmo agente. `POST /perguntas` recebe `{"pergunta": "..."}` e devolve a pergunta e a resposta; `GET /saude` responde sem chamar o modelo, com `ok` e o provedor/modelo em uso. Os blocos 1 a 3 do `Dockerfile` são os mesmos, com `api.py` acrescentado ao `COPY`; o bloco 4 troca o programa que roda e termina por um servidor que fica de pé:

```
EXPOSE 8000
CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
docker run --rm -p 8000:8000 --env-file .env auditor-de-despesas-web:1.0.0
curl -X POST localhost:8000/perguntas -H 'content-type: application/json' \
     -d '{"pergunta": "A despesa D-4613 está dentro da política?"}'
```

`--host 0.0.0.0`	com o padrão `127.0.0.1`, o uvicorn escuta só no *loopback* do container, e a porta publicada não alcança nada
`EXPOSE` × `-p`	`EXPOSE` documenta a porta; quem a liga à máquina é o `-p` do `docker run`
`api:app`	o uvicorn importa o módulo pelo nome: `02-api.py` não seria importável
síncrona	a conexão fica aberta enquanto o agente trabalha — serve a um agente que responde em segundos

O segredo: apagar não apaga — duas vezes

No git — o `.env` foi commitado, e apagado depois:

```
$ git log --all -S "_API_KEY=" --oneline
3f9a2c1 ajusta configuração
```

Na imagem — um `Dockerfile` que parece seguro e não é (saída ilustrativa):

```
COPY .env .
RUN pip install -r requirements.txt
RUN rm .env
```

```
$ docker history auditor-de-despesas:inseguro
CREATED BY                                SIZE
RUN rm .env                               0B
RUN pip install -r requirements.txt       412MB
COPY .env .                               156B
```

A camada do `COPY` é imutável; o `rm` só acrescenta uma camada que esconde o arquivo. `docker save` exporta as camadas, e a do `COPY` contém o `.env` íntegro.

A correção é a mesma nas duas formas: revogar a chave no provedor. Reescrever o histórico ou reconstruir a imagem não alcança clones, *forks*, registros e cópias que já existem.

Parte 4

O container é efêmero

O sistema de arquivos nasce da imagem a cada execução

O que o processo grava dentro do container some quando ele termina — com `--rm`, imediatamente. Demonstração, trocando o programa do `ENTRYPOINT` por um *shell*:

```
$ docker run --rm --entrypoint sh auditor-de-despesas:1.0.0 -c "date > /app/estado.txt; cat /app/estado.txt"
Thu Oct 15 21:10:42 UTC 2026

$ docker run --rm --entrypoint sh auditor-de-despesas:1.0.0 -c "cat /app/estado.txt"
cat: /app/estado.txt: No such file or directory
```

Os dois containers nasceram da mesma imagem, e a imagem não contém nada do que o primeiro escreveu: a gravação em `/app` foi para a camada gravável do primeiro container, apagada com ele.

A propriedade não é defeito: a imagem é imutável e o container é descartável **por desenho**. É isso que permite trocar `1.0.0` por `1.1.0` sem cerimônia — e é isso que obriga o estado a ter **endereço declarado**.

Onde mora o estado: três saídas, cada uma com preço

Saída	Como	Preço
reconstruir	na partida; ou no build, numa camada da imagem	partida lenta e o custo de embutir o corpus pago a cada execução; no build, imagem maior e um build a cada mudança do corpus
externalizar	volume montado com `-v`, banco de dados, serviço separado	um componente a mais, com disponibilidade e conta próprias
aceitar a perda, e declarar	nada é gravado fora do container	legítimo para o que é derivado; inaceitável para o que não é

Estado	Derivado de	Saídas admissíveis
índice da aula de RAG	um corpus que existe em outro lugar	as três; reconstruir no build serve a índice estático
memória da aula de memória	**nada** — é o que o sistema aprendeu em execução	só externalizar

O volume dá ao estado um endereço fora do container

```
$ docker run --rm -v auditor-estado:/tmp/estado --entrypoint sh auditor-de-despesas:1.0.0 -c "date > /tmp/estado/estado.txt"

$ docker run --rm -v auditor-estado:/tmp/estado --entrypoint sh auditor-de-despesas:1.0.0 -c "cat /tmp/estado/estado.txt"
Thu Oct 15 21:12:07 UTC 2026
```

- `-v nome:/caminho` monta um **volume nomeado**, gerido pelo Docker e independente de qualquer container: sobrevive ao `--rm` e ao container seguinte.
- O caminho de montagem é um diretório do container; o que se grava abaixo dele vai para o volume, e não para a camada gravável do container.
- O volume é a forma local de **externalizar**. O preço reaparece em outra escala: o estado deixa de viajar com a imagem, e outra máquina com a mesma *tag* não o tem.

O banco como estado externalizado: `03-agente-web-banco/`

A política de reembolso sai do dicionário no código e vai para a tabela `politica` do Postgres. A imagem da API é a da `02-agente-web/`, sem mudança: o endereço do banco chega por `DATABASE_URL`, e quem sobe o banco ao lado é o `docker-compose.yml`.

```
services:
  banco:
    image: postgres:16.10
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - dados:/var/lib/postgresql/data
      - ./banco:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 2s
      timeout: 3s
      retries: 15
  agente:
    build: .
    image: auditor-de-despesas-banco:1.0.0
    env_file: .env
    environment:
      DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@banco:5432/${POSTGRES_DB}
    ports:
      - "8000:8000"
    depends_on:
      banco:
        condition: service_healthy
volumes:
  dados:
```

Quatro decisões do `docker-compose.yml`

Decisão	Mecanismo	Consequência
o banco nasce de um script	`banco/01-politica.sql` montado em `/docker-entrypoint-initdb.d/`; a imagem do Postgres o executa **uma vez**, com o volume vazio	mudar o script não altera um banco existente; `docker compose down -v` apaga o volume e o script roda de novo
o estado tem endereço declarado	volume nomeado `dados` em `/var/lib/postgresql/data`	`docker compose down` e `up` recriam os containers sem perder a política
a API alcança o banco pelo nome do serviço	o compose cria uma rede própria e resolve `banco` para o container certo: `banco:5432`, não `localhost`	o banco não tem `ports`: só é alcançável de dentro da rede do compose
"de pé" não é "pronto"	`healthcheck` com `pg_isready` e `depends_on` com `condition: service_healthy`	a API só parte quando o banco aceita conexões, e não apenas quando o processo dele iniciou

A ferramenta `consultar_politica` faz uma **consulta parametrizada** — a categoria vem do modelo, que não é fonte confiável:

```
linha = conexao.execute("SELECT teto FROM politica WHERE categoria = %s", (categoria,)).fetchone()
```

O `GET /saude` passa a informar também se o banco responde (`"banco": true`), ainda sem chamar o modelo.

Execuções longas: além da API síncrona

Conceito, sem código nesta aula. A API da `02-agente-web/` é síncrona e serve a um agente de poucos passos; para execuções longas, ela não serve: uma execução de doze passos leva minutos, e nenhum intermediário HTTP — balanceador, *proxy*, *gateway* — mantém uma conexão ociosa por esse tempo. O princípio comum é **desacoplar a aceitação do pedido da entrega do resultado**.

Padrão	Mecanismo	Quando
fila com trabalhador	o pedido entra numa fila e recebe um identificador; o cliente consulta o estado	execução longa, cliente que volta depois
webhook	o serviço chama o endereço registrado pelo cliente ao terminar	integração entre sistemas
SSE (*Server-Sent Events*)	eventos de progresso pela mesma conexão, do servidor ao cliente	pessoa diante da tela

O critério é a **duração** da execução e a **presença de alguém esperando**. Os três são os modos de entrega que o A2A padroniza, retomados na aula de multiagente. O que os viabiliza é a **execução durável** da aula de *frameworks* e orquestração: com o container descartável, o checkpoint deixa de ser conveniência e vira requisito.

A imagem entregue, e a superfície que ela cria

Existe agora uma imagem que qualquer máquina com Docker executa igual: identificada pela *tag*, com o segredo fora do repositório e fora das camadas, e o estado com endereço declarado. A mesma imagem define a superfície de que parte a aula de segurança:

Superfície	O que expõe
imagem com dependências de terceiros	a base `python:3.11.13-slim-bookworm` e nove pacotes fixados — cada um com suas próprias dependências e vulnerabilidades
processo como `root`	o padrão do Docker: o processo tem, dentro do container, todo o privilégio que o isolamento permite
segredo em variável de ambiente	a chave legível por qualquer código que rode no processo — inclusive o de uma dependência
um agente que age	ferramentas chamadas por decisão do modelo, a partir de um objetivo que vem de fora
porta publicada (`02-agente-web/`, `03-agente-web-banco/`)	o `-p` e o `ports` aceitam pedidos de quem alcançar a máquina
banco ao alcance de uma ferramenta (`03-agente-web-banco/`)	credenciais no mesmo `.env` e texto do modelo levado a uma consulta SQL — parametrizada